

REACT

DATA ARCHITECTURE

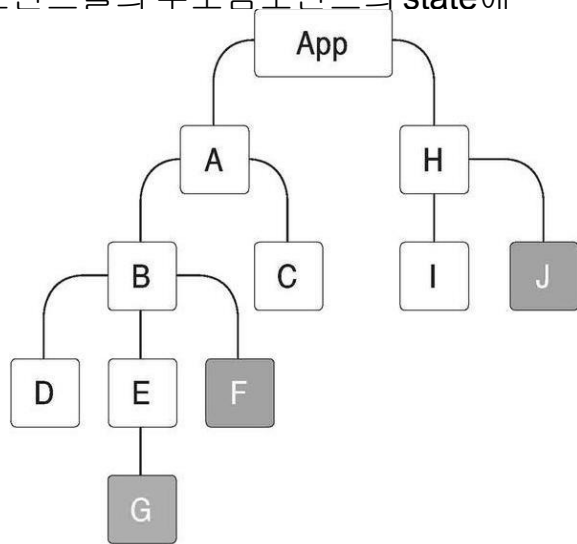
단방향 데이터 흐름

- React의 단방향 데이터 흐름 구조

자식컴포넌트는 부모컴포넌트에게 데이터를 줄수 없고 부모만 자식에게 데이터를 전달 할 있음. [property(속성)를 통해]

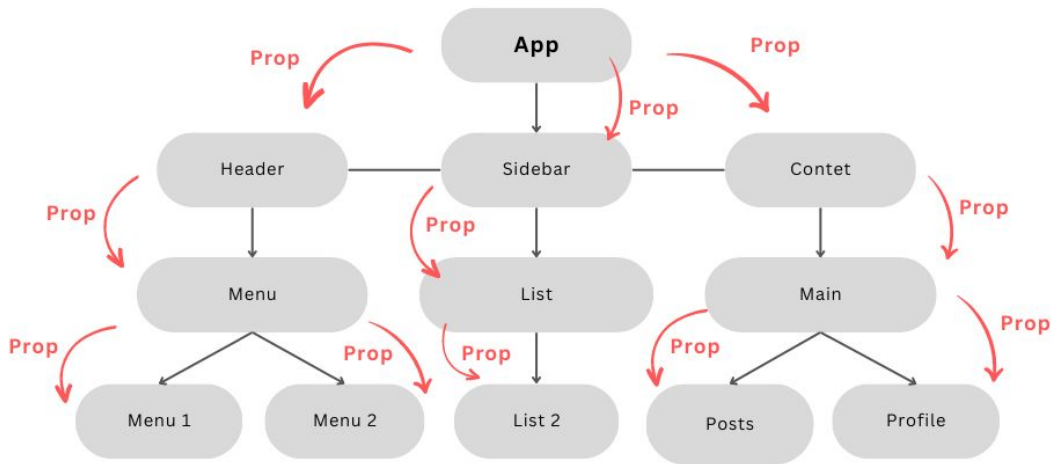
- 또한, 자식들 끼리 서로 데이터를 주고 받을 수도 없음

- 그렇기에 컴포넌트들 간에 같은 데이터를 사용하고자 한다면 그 컴포넌트들의 부모컴포넌트의 **state**에 데이터를 저장하고 자식들은 **props**를 통해 전달받아 값을 공유함.



단방향 데이터 흐름 문제

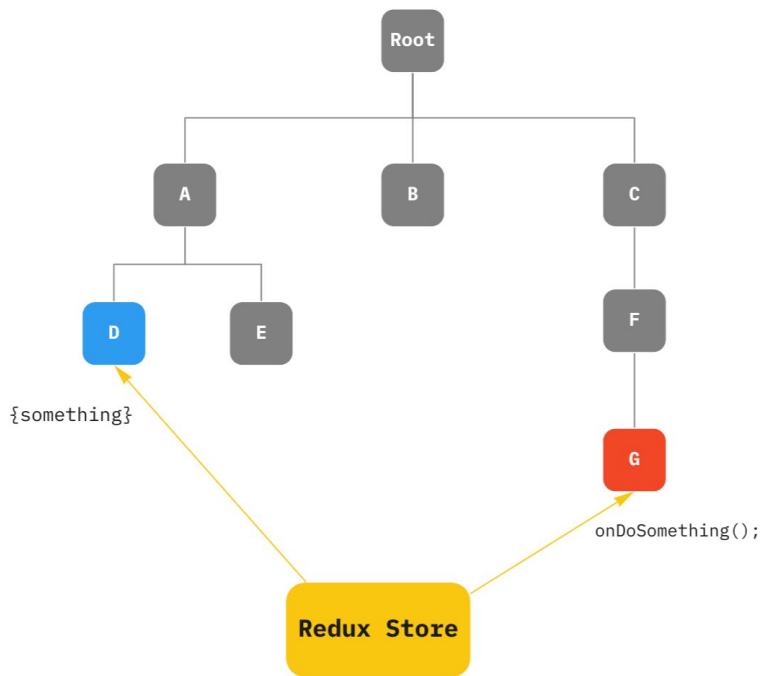
- 하지만, 앱의 규모가 커져 컴포넌트의 중첩이 많아져서 부모-자식-손자-증손자-고손자..... 등으로 컴포넌트의 단계가 많아지면 부모의 **state**값을 고손자 까지 전달하기 위해 **props**를 통해 전달->전달->전달->전달 을 해야 하는 상황이 생김.
- 초기에는 상관없으나 조금의 변경상황이 생겼을 때 그 수정과정이 매우 번거롭고 난해하며 어느 단계에서 데이터가 변경되어 자식->자식->자식으로 전달되었는지 파악하기 어려움



앱에서 **Data**관리를 어떻게 할 것인가에 대한 고민이 필요함.

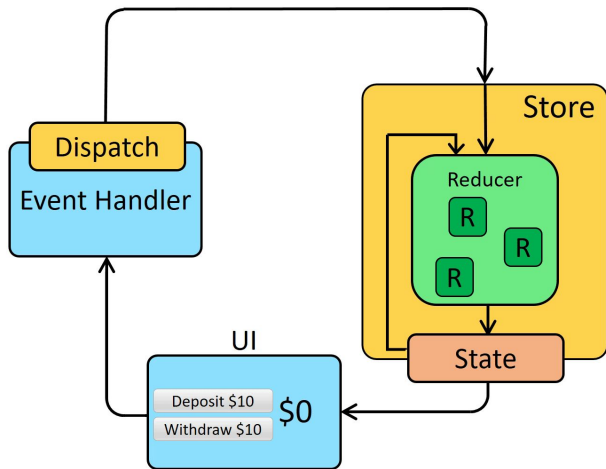
Flux 패턴의 도입

- 이 고민의 결과로 **Flux** 패턴이 도입됨 (일종의 전역변수저장 공간(**Store**라고 부름) 1개를 만들고 모든 컴포넌트들이 이 공간을 접속하여 데이터를 취급하는 구조의 패턴)

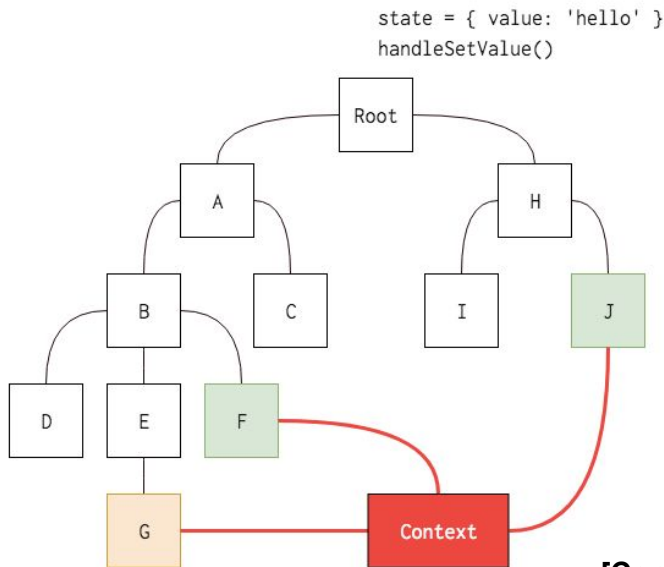


Flux 패턴 라이브러리 [Redux , Context API]

- 이 Flux패턴을 구현한 라이브러리가 **Redux** 임.
- **Redux** 라이브러리는 매우 유명하고 많이 사용되지만 초기학습과정이 어렵다는 단점이 있음.
- React에서는 이 **Redux**을 대신하기 위해 **Context API**를 개선하였음.



[redux 동작흐름]



[Context API]

Flux 패턴 라이브러리 [Redux , Context API]

- 이 Flux패턴을 구현한 라이브러리가 **Redux** 임.
- **Redux** 라이브러리는 매우 유명하고 많이 사용되지만 초기학습과정이 어렵다는 단점이 있음.
- React에서는 이 **Redux**을 대신하기 위해 **Context API**를 개선하였음.

